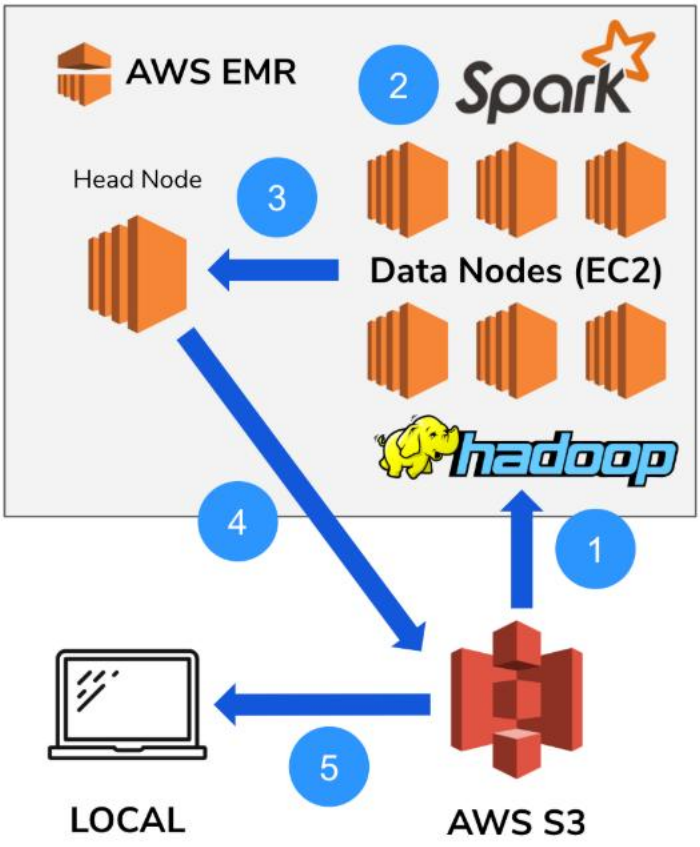
REPORT

BIG DATA WRANGLING WITH GOOGLE BOOKS NGRAMS



Author: Ryan Austin, Student

Institution: BrainStation Data Science Bootcamp

Submitted: March 14, 2026

Table of Contents

Introduction 3
Objective 3
Dataset 3
Deliverables 3
Big Data Analysis Workflow 4
Pulling Data from S3 into HDFS 4
Analyzing and Filtering the Data Using PySpark 4
Collecting the Directory Contents into a Single File 8
Assessing the data from a Local Machine 9
Plotting the Number of Occurrences of Data over the Years 10
Hadoop vs. Spark as Distributed File Systems 11
APPENDIX I 12
APPENDIX II 17

Introduction

Objective

To load, filter, and visualize a large real-world dataset in a cloud-based distributed computing environment using Hadoop, Spark, and the S3 filesystem.

Dataset

The Google Ngrams dataset was created by Google's research team by analyzing all the content in Google Books. The digitized texts represent approximately four percent of all books ever printed and span from the 1800s into the 2000s. The dataset is hosted in a public S3 bucket as part of the Amazon S3 Open Data Registry. The data was converted to CSV and hosted on a public S3 bucket which may be accessed at:

s3://brainstation-dst/eng_1M_1gram.csv.

Deliverables

1. Project report
2. Jupyter notebook for PySpark analysis via EMR: Ryan_Austin_Unit4_EMR_PySpark
3. Jupyter notebook for analysis on local machine: Ryan_Austin_Unit4_Local

Big Data Analysis Workflow

As part of the workflow, the source data was filtered and reduced to a manageable size. After extracting the data from the cloud and processing it using Big Data tools, analysis was done with a local machine.

WORKFLOW STEPS

1. Pull data from (public) S3 bucket into HDFS
2. Analyze and filter data using PySpark
3. Collect filtered data to head node using *getmerge*
4. Write data to personal S3 bucket
5. Read data from S3 bucket for analysis on local machine

Pulling Data from S3 into HDFS

From HDFS command line prompt, the directory tree data where the file pulled from S3 would be stored was created:

hdfs fs -mkdir -p /user/hadoop/eng_data_1gram

Verify that the directory was created with the command:

hadoop fs -ls

The command to pull the file was:

hadoop distcp s3://brainstation-dsft/eng_1M_1gram.csv /user/hadoop/eng_data_1gram/eng_1M_1gram.csv

Verify that the file was transferred with:

hadoop fs -ls/user/hadoop/eng_data_1gram/

Note: The *hadoop distcp* command was used to more efficiently transfer the large dataset via distributed processing.

Analyzing and Filtering the Data Using PySpark

The PySpark analysis of the data was done with a JupyterHub notebook. Note that before trying to access the notebook the EMR cluster must be in the "waiting" state. The notebook was accessed from a browser by entering the local host https://localhost:9995 in

the browser url field. Various security warnings came up. It was still possible to proceed. At the sign-in prompt, the default username and password were entered.

In the Jupyter GUI create a new PySpark notebook. To ensure that the Spark configurations are set correctly, enter the following configuration code in the first cell of the notebook:

```
In [33]: # Ensure that the Spark configuration is set correctly
         %%configure -f
         {
             "conf": {
                 "spark.pyspark.python": "python3",
                 "spark.pyspark.virtualenv.enabled": "true",
                 "spark.pyspark.virtualenv.type":"native",
                 "spark.pyspark.virtualenv.bin.path":"/usr/bin/virtualenv"
             }
         }
```

```
Starting Spark application
```

ID	YARN Application ID	Kind	State	Spark UI	Driver log	User	Current session?
2	application_1773441513972_0007	pyspark	idle	Link	Link	None	✓

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

SparkSession available as 'spark'.

Current session configs: {'conf': {'spark.pyspark.python': 'python3', 'spark.pyspark.virtualenv.enabled': 'true',
'spark.pyspark.virtualenv.type': 'native', 'spark.pyspark.virtualenv.bin.path': '/usr/bin/virtualenv'}, 'proxyUser': 'jovyan',
'kind': 'pyspark'}
```

ID	YARN Application ID	Kind	State	Spark UI	Driver log	User	Current session?

The data science and plotting packages were then installed:

```
In [2]: %%info
```

```
Current session configs: {'conf': {'spark.pyspark.python': 'python3', 'spark.pyspark.virtualenv.enabled': 'true',
'spark.pyspark.virtualenv.type': 'native', 'spark.pyspark.virtualenv.bin.path': '/usr/bin/virtualenv'}, 'proxyUser': 'jovyan',
'kind': 'pyspark'}

No active sessions.
```

```
In [3]: # Install data science & plotting packages

        sc.install_pypi_package("pandas==1.0.5")
        sc.install_pypi_package("matplotlib==3.7.5")
```

```
Starting Spark application
```

ID	YARN Application ID	Kind	State	Spark UI	Driver log	User	Current session?
1	application_1773441513972_0003	pyspark	idle	Link	Link	None	✓

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

SparkSession available as 'spark'.

FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

Collecting pandas==1.0.5
  Downloading pandas-1.0.5.tar.gz (5.0 MB)
```

The necessary libraries were then installed, and the data was read from the S3 bucket into a new data frame with the command:

data_df = spark.read.csv("/directory/path/file.csv", header=True).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…
```

```
# Read the data from HDFS.
data_df = spark.read.csv('/user/hadoop/eng_data_1gram/eng_1M_1gram.csv', header=True)
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…
```

```
#  Print out the rows to get the shape of the dataframe
nrows = data_df.count()
print("There are", nrows, "rows in the data frame.")
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

There are 261823225 rows in the data frame.
```

```
# Determine the number of columns and print out the shape of the data frame.
ncols = len(data_df.columns)
data_shape = (nrows, ncols)
print("The shape (rows, columns) of the data frame is", data_shape, ".")
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

The shape (rows, columns) of the data frame is (261823225, 5) .
```

The screenshot above also shows that to get the shape of the data frame, the number of rows, then the number of columns was calculated, then the shape was defined as the number of rows and columns.

The schema of the data frame was also generated:

```
# Show the schema of the data frame.
data_df.printSchema()
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

root
 |-- token: string (nullable = true)
 |-- year: string (nullable = true)
 |-- frequency: string (nullable = true)
 |-- pages: string (nullable = true)
 |-- books: string (nullable = true)
```

```
data_df.show()
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

+---------+----+---------+-----+-----+
|    token|year|frequency|pages|books|
+---------+----+---------+-----+-----+
|inGermany|1927|        2|    2|    2|
|inGermany|1929|        1|    1|    1|
|inGermany|1930|        1|    1|    1|
|inGermany|1933|        1|    1|    1|
```

A new data frame of data filtered from the original data frame was generated:

```
# Q 4b: Create a new DataFrame from a query using Spark SQL, filtering to include only the rows where the token is "data"
# and describe the new dataset

from pyspark.sql.functions import col

fltrd_data_df = data_df.filter(col("token") == "data")
fltrd_data_df.count()

# Alternate method:
#fltrd_data_df2 = data_df.filter(data_df.token == "data")
#fltrd_data_df2.count()
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

316
```

The filtering criteria was that the entries in the “Token” field must be “data”.

The shape of the new data frame was also generated:

```
# Q4b cont'd: Describe the filtered data set.
fltrd_nrows = fltrd_data_df.count()
fltrd_ncols = len(fltrd_data_df.columns)
fltrd_data_shape = (fltrd_nrows, fltrd_ncols)
print("The shape (rows, columns) of the filtered data frame is", fltrd_data_shape, ".")
print("The shape (rows, columns) of the original data frame was", data_shape, ".")
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…

The shape (rows, columns) of the filtered data frame is (316, 5) .
The shape (rows, columns) of the original data frame was (261823225, 5) .
```

The filtered data frame was written back to an HDFS data frame:

```
# Q4c: Write the filtered data back to a directory in the HDFS from Spark
fltrd_data_df.write.csv('/user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram', header=True)
```

```
FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=Layout(height='25px', width='50%'),…
```

The output was the original CSV file partitioned into two CSV files in the */user/Hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram* directory (see screenshots below).

NOTE: Running the code above resulted in the “Permission denied” error. An investigation revealed that the reason was because “Livy” was the owner and “Livy” did not have write permission. The following code had to be run in GItBash:

hdfs dfs -chown livy:hdfsadmingroup /user/Hadoop/eng_fltrd_data_1gram

for the file to be written and appear in the assigned directory.

Finally, the Spark session was closed gracefully with the *spark.stop()* command.

The transfer of the file in partitions to HDFS was verified with the *hadoop fs -ls* command:

```
[hadoop@ip-172-31-20-124 ~]$ hadoop fs -ls /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram
Found 3 items
-rw-r--r--   1 livy hdfsadmingroup          0 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/_SUCCESS
-rw-r--r--   1 livy hdfsadmingroup         33 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00000-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
-rw-r--r--   1 livy hdfsadmingroup       7305 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00023-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
[hadoop@ip-172-31-20-124 ~]$
```

The content of the two file partitions was verified with the *hdfs dfs -cat* command:

```
[hadoop@ip-172-31-20-124 ~]$ hadoop fs -ls /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram
Found 3 items
-rw-r--r--   1 livy hdfsadmingroup          0 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/_SUCCESS
-rw-r--r--   1 livy hdfsadmingroup         33 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00000-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
-rw-r--r--   1 livy hdfsadmingroup       7305 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00023-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
[hadoop@ip-172-31-20-124 ~]$ hdfs dfs -cat /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00000-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
token,year,frequency,pages,books
[hadoop@ip-172-31-20-124 ~]$
```

The first partition only had field names. The second partition had the data.

```
[hadoop@ip-172-31-20-124 ~]$ hadoop fs -ls /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram
Found 3 items
-rw-r--r--   1 livy hdfsadmingroup          0 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/_SUCCESS
-rw-r--r--   1 livy hdfsadmingroup         33 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00000-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
-rw-r--r--   1 livy hdfsadmingroup       7305 2026-03-14 00:38 /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00023-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
[hadoop@ip-172-31-20-124 ~]$ hdfs dfs -cat /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00000-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
token,year,frequency,pages,books
[hadoop@ip-172-31-20-124 ~]$ hdfs dfs -cat /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram/part-00023-79657fc3-4c48-4e7b-a69c-e2ac33908142-c000.csv
token,year,frequency,pages,books
data,1584,16,14,1
data,1614,3,2,1
data,1627,1,1,1
data,1631,22,18,1
data,1637,1,1,1
data,1638,2,2,1
data,1640,1,1,1
data,1642,1,1,1
data,1644,4,4,1
data,1647,1,1,1
data,1651,1,1,1
data,1674,1,1,1
data,1690,1,1,1
data,1693,1,1,1
data,1697,1,1,1
data,1699,1,1,1
data,1700,1,1,1
data,1701,11,11,2
data,1702,1,1,1
data,1703,1,1,1
data,1705,3,3,1
data,1707,2,2,2
data,1708,1,1,1
data,1709,1,1,1
data,1711,4,4,4
data,1713,40,28,1
data,1714,3,3,3
data,1715,1,1,1
data,1717,1,1,1
data,1718,1,1,1
data,1720,7,7,3
data,1722,1,1,1
data,1723,1,1,1
data,1724,2,2,2
data,1725,1,1,1
data,1726,1,1,1
data,1727,11,11,4
```

Collecting the Directory Contents into a Single File

The partitioned files were merged in HDFS and transferred to the local directory with the command: *hdfs dfs -getmerge /Hadoop/directory/path merged_filename.csv*

The file transfer to the local directory was verified with the command: *ls -ltr*.

The personal bucket options in S3 were verified with the command: *aws s3 ls*

The merged file was transferred to a personal S3 bucket with the command:

aws s3 cp filename.csv s3://personal-s3-bucket/filename.csv

```
[hadoop@ip-172-31-20-124 ~]$ hdfs dfs -getmerge /user/hadoop/eng_fltrd_data_1gram/fltrd_eng_1M_1gram merged_fltrd_eng_1M_1gram.csv
[hadoop@ip-172-31-20-124 ~]$ ls -ltr
total 8
-rw-r--r--. 1 hadoop hadoop 7338 Mar 14 01:45 merged_fltrd_eng_1M_1gram.csv
[hadoop@ip-172-31-20-124 ~]$ aws s3 ls
2026-03-03 03:06:38 aws-logs-409257555249-us-east-2
2026-03-03 01:48:06 raustin-bstn-bucket
[hadoop@ip-172-31-20-124 ~]$ aws s3 cp merged_fltrd_eng_1M_1gram.csv s3://raustin-bstn-bucket/merged_fltrd_eng_1M_1gram.csv
upload: ./merged_fltrd_eng_1M_1gram.csv to s3://raustin-bstn-bucket/merged_fltrd_eng_1M_1gram.csv
[hadoop@ip-172-31-20-124 ~]$
```

Assessing the data from a Local Machine

To read the data into a Panda data frame on the local machine, a user must first configure their credentials using the *aws configure* command at the command prompt. The process involved setting up an AWS Access Key ID and an AWS Secret Access Key, and adding “AmazonS3FullAccess” in the IAM section of AWS.

```
(base)
raust@RSpacer2024 MINGW64 ~
$ conda deactivate

raust@RSpacer2024 MINGW64 ~
$ conda activate cloud_env
(cloud_env)
raust@RSpacer2024 MINGW64 ~
$ aws --version
aws-cli/1.35.1 Python/3.11.5 Windows
(cloud_env)
raust@RSpacer2024 MINGW64 ~
$ aws configure
AWS Access Key ID [None]: AKIAV6SNGW
AWS Secret Access Key [None]: DaR5FO
Default region name [None]: us-east-
Default output format [None]: json
(cloud_env)
raust@RSpacer2024 MINGW64 ~
$ aws configure list
      Name                    Value
      ----                    -----
   profile                <not set>
access_key     *****************VCAK
secret_key     *****************HqyQ
    region                us-east-1
(cloud_env)
raust@RSpacer2024 MINGW64 ~
$ |
```

In the Jupyter notbook, dependencies had to be installed to read from the S3 bucket:

```
In [1]: # Install dependencies in order to read data from S3 in Python
        pip install pandas s3fs

Requirement already satisfied: pandas in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (2.2.2)
Requirement already satisfied: s3fs in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (2024.9.0)
Requirement already satisfied: numpy>=1.23.2 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from pandas) (1.24.
3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from panda
s) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from pandas) (2023.
3)
Requirement already satisfied: aiobotocore<3.0.0,>=2.5.4 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from s3f
s) (2.15.1)
Requirement already satisfied: fsspec==2024.9.0.* in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from s3fs) (202
4.9.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from s3
fs) (3.10.5)
Requirement already satisfied: botocore<1.35.24,>=1.35.16 in c:\users\raust\anaconda3\envs\cloud_env\lib\site-packages (from ai
obotocore<3.0.0,>=2.5.4->s3fs) (1.35.23)
```

After ensuring all permissions were met, the code was straightforward:

```
In [7]: # Import necessary modules
        import pandas as pd
        import matplotlib.pyplot as plt
```

```
In [8]: # Read the file content
        filtered_data_df = pd.read_csv("s3://raustin-bstn-bucket/merged_fltrd_eng_1M_1gram.csv")
        print(filtered_data_df.head())

           token  year  frequency  pages  books
        0  token  year  frequency  pages  books
        1   data  1584         16     14      1
        2   data  1614          3      2      1
        3   data  1627          1      1      1
        4   data  1631         22     18      1
```

Plotting the Number of Occurrences of Data over the Years

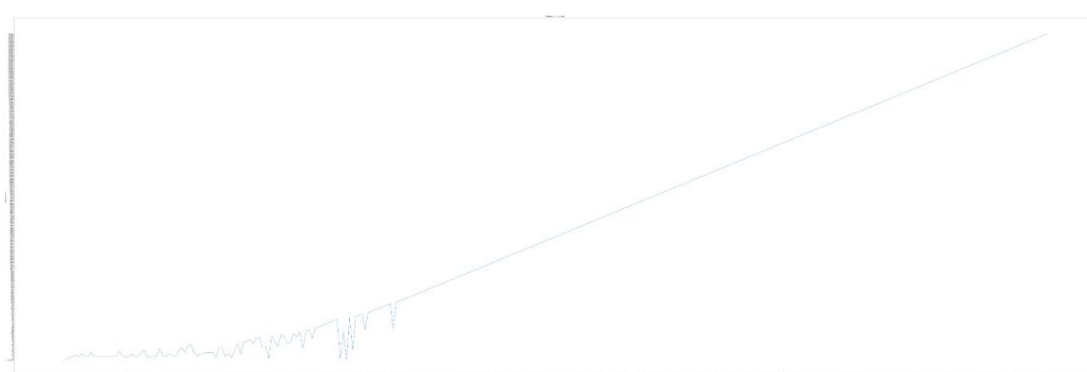
Both the data range and domain were too great be plotted in its entirety on one page without overlap. The *%matplotlib notebook* command was executed first so that the plot generated would be interactive in terms of zoomability and pan-ability. The user will have to zoom in enough for the values and labels to be differentiable and legible, then pan over sections of the plot to interpret it. The code was otherwise straightforward.

```
In [20]: # This code must be added in order to read the labels and axes of the plot via zooming and panning.
         # The labels and axes would be illegible if the entire plot was viewed all at once.
         %matplotlib notebook
```

```
In [24]: # Plot the frequency over the years.
         plt.figure(figsize=(150, 50))
         x = filtered_data_df["year"]
         y = filtered_data_df["frequency"]
         plt.plot(x, y)
         plt.xlabel("Years")
         plt.ylabel("Frequency Count")
         plt.title("Frequency Count by Year")
         plt.show()

         <IPython.core.display.Javascript object>
```

A zoomed-out view of the entire plot:



Hadoop vs. Spark as Distributed File Systems

Hadoop and Spark are both open-source big data frameworks from the Apache Software Foundation. Hadoop is designed for distributed batch processing using the Hadoop Distributed File System (HDFS) and MapReduce programming model. Spark is built for in-memory distributed processing using Resilient distributed Datasets (RDDs).

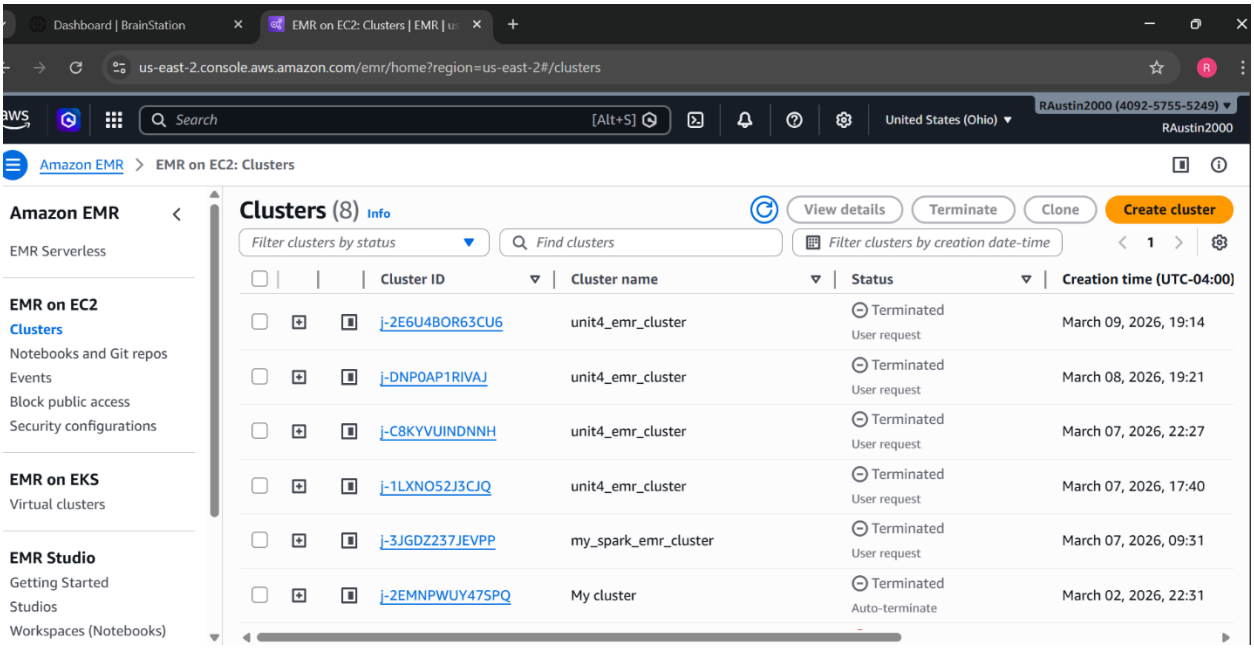
Two advantages for Hadoop are that it is more cost-effective and scalable for large datasets, and that it offers more robust security (LDAP, ACLs, encryption). Spark scaling requires more RAM, increasing cost. Spark's security is basic unless it is integrated with Hadoop.

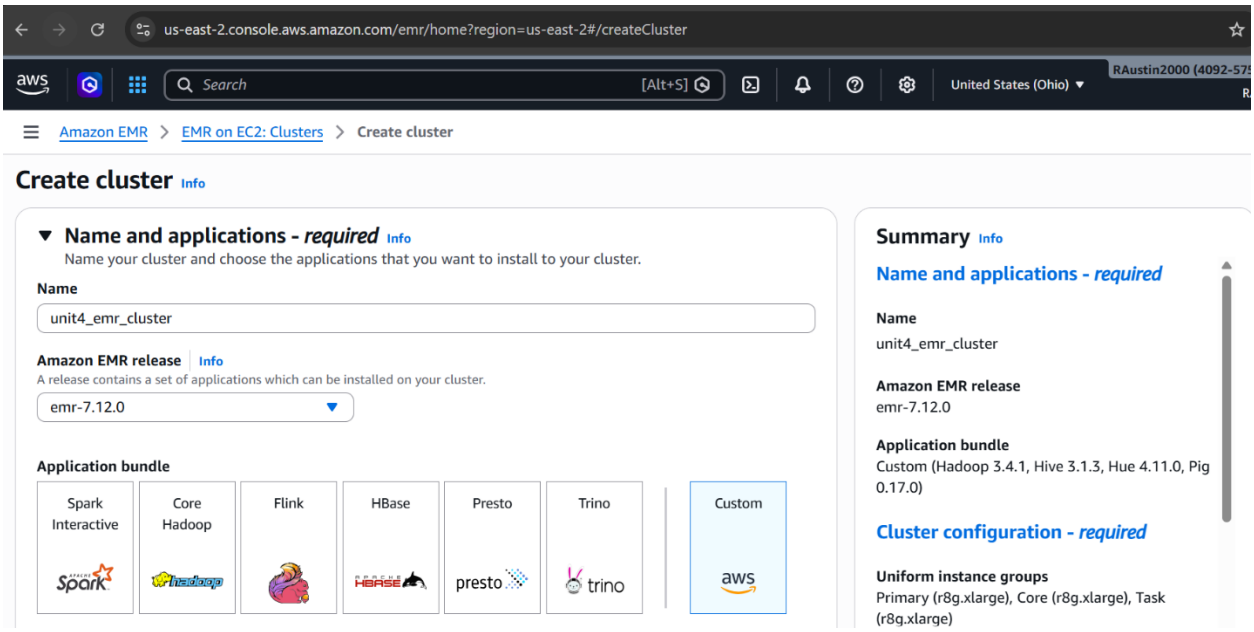
Two advantages for Spark are that it is faster for iterative and interactive workloads (up to 100x faster than Hadoop MapReduce for certain workloads) and that it has a built-in machine learning library MLlib. Hadoop needs external libraries (e.g., Mahout).

Hadoop Data File System (HDFS) stores data by splitting large files into smaller, fixed-size chunks called blocks. These blocks are then distributed across multiple DataNodes, which are responsible for storing the actual data blocks. A coordinating node called the NameNode acts as the master server, managing the file system namespace, including the metadata, directory structure, and the mapping of files to the underlying storage blocks. HDFS is designed to be fault-tolerant, ensuring that the failure of a single node does not result in data loss.

APPENDIX I

Screenshots for creating a new EMR cluster on AWS for using Spark and EMR notebooks.





us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster

Amazon EMR > EMR on EC2: Clusters > Create cluster

- [ ] AmazonCloudWatchAgent 1.300032.2
- [ ] HCatalog 3.1.3
- [x] Hue 4.11.0
- [x] Livy 0.8.0
- [ ] Pig 0.17.0
- [ ] TensorFlow 2.19.0
- [ ] Zeppelin 0.11.1
- [ ] Flink 1.20.0
- [x] Hadoop 3.4.1
- [ ] JupyterEnterpriseGateway 2.6.0
- [ ] Oozie 5.2.1
- [ ] Presto 0.287
- [ ] Tez 0.10.2
- [ ] ZooKeeper 3.9.3
- [ ] HBase 2.6.2
- [x] Hive 3.1.3
- [x] JupyterHub 1.5.0
- [ ] Phoenix 5.2.1
- [x] Spark 3.5.6
- [ ] Trino 476

AWS Glue Data Catalog settings

Use the AWS Glue Data Catalog to provide an external metastore for your application.

- [ ] Use for Hive table metadata
- [ ] Use for Spark table metadata

Operating system options Info

- (•) Amazon Linux release
- ( ) Custom Amazon Machine Image (AMI)
- [x] Automatically apply latest Amazon Linux updates

Summary Info

Name and applications - *required*

Name
unit4_emr_cluster

Amazon EMR release
emr-7.12.0

Application bundle
Custom (Hadoop 3.4.1, Hive 3.1.3, Hue 4.11.0, JupyterHub 1.5.0, Livy 0.8.0, Spark 3.5.6)

Cluster configuration - *required*

Uniform instance groups
Primary (r8g.xlarge), Core (r8g.xlarge), Task (r8g.xlarge)

Cluster scaling and provisioning - *required*

us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster

Amazon EMR > EMR on EC2: Clusters > Create cluster

Lowest Spot price: $0.063 (us-east-2c)

▸ **Node configuration - *optional***

Add task instance group

You can add up to 48 more task instance groups.

EBS root volume

EBS root volume applies to the operating systems and applications that you install on the cluster. EBS root volume ratio constraints

Size (GiB)	IOPS	Throughput (MiB/s)
15	3000	125
15 - 100 GiB per volume General Purpose SSD (gp3)	3000 - 16000 IOPS per volume. Choose a maximum ratio of 500:1 between IOPS and volume size.	125 - 1000 MiB/s per volume. Choose a maximum ratio of 0.25:1 between throughput and IOPS.

Summary Info

Uniform instance groups
Primary (r8g.xlarge), Core (r8g.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration
Core size: 1 instance

Networking - *required*

VPC
vpc-0786984ef...

Subnet
subnet-0372e2...

Primary node security group
sg-025ad1917c...

Core node security group
sg-0ef6de7eb5...

aws | Search [Alt+S] | United States (Ohio) | RAustin2000 (4092-5

Amazon EMR > EMR on EC2: Clusters > Create cluster

Cluster scaling and provisioning - *required* Info

Choose how Amazon EMR should size your cluster.

Choose an option

- (●) **Set cluster size manually** — Use this option if you know your workload patterns in advance.
- ( ) **Use EMR-managed scaling** — Monitor key workload metrics so that EMR can optimize the cluster size and resource utilization.
- ( ) **Use custom automatic scaling** — To programmatically scale core and task nodes, create custom automatic scaling policies.

Provisioning configuration

Set the size of your core instance group. Amazon EMR attempts to provision this capacity when you launch your cluster.

Name	Instance type	Instance(s) size	Use Spot purchasing option
Core	r8g.xlarge	2	☐

Networking - *required* Info

Summary Info

Uniform instance groups
Primary (r8g.xlarge), Core (r8g.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration
Core size: 2 instances

Networking - *required*

VPC
vpc-0786984ef...

Subnet
subnet-0372e2...

Primary node security group
sg-025ad1917c...

Core node security group
sg-0ef6de7eb5...

us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster

aws | Search [Alt+S] | United States (Ohio) | RAustin2000 (4092-575

Amazon EMR > EMR on EC2: Clusters > Create cluster

Cluster termination and node replacement Info

Choose termination settings and protect your cluster from accidental shutdown.

Termination option

- ( ) Manually terminate cluster
- ( ) Automatically terminate cluster after last step ends
- (●) Automatically terminate cluster after idle time (Recommended)

Idle time

Enter the time until your cluster terminates.

0 days | 04:00:00

Choose a time that is greater than 1 minute (00:01:00) and less than 7 days. The time is in hh:mm:ss (24-hour) format.

☐ Use termination protection
Protects your cluster from accidental termination. If on, you must first turn off protection to terminate the cluster. We recommend turning on termination protection for your long running clusters.

Unhealthy node replacement - *new* Info

(●) Turn on
Amazon EMR gracefully stops processes on unhealthy nodes to minimize data loss and job interruptions. It quickly replaces unhealthy nodes with new EC2 instances to keep your jobs running smoothly.

( ) Turn off

Summary Info

Uniform instance groups
Primary (r8g.xlarge), Core (r8g.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration
Core size: 2 instances

Networking - *required*

VPC
vpc-0786984ef...

Subnet
subnet-0372e2...

Primary node security group
sg-025ad1917c...

Core node security group
sg-0ef6de7eb5...

us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster

aws | Search [Alt+S] | United States (Ohio) | RAustin2000 (4092-5755

Amazon EMR > EMR on EC2: Clusters > Create cluster

Security configuration and EC2 key pair Info

Choose a security configuration or create a new one that you can reuse with other clusters.

Security configuration

Select your cluster encryption, authentication, authorization, and instance metadata service settings.

Choose a security configuration | Browse | Create security configuration

Amazon EC2 key pair for SSH to the cluster Info

aws_cloud | Browse | Create key pair

Identity and Access Management (IAM) roles - *required* Info

Choose or create a service role and instance profile for the EC2 instances in your cluster.

Amazon EMR service role Info

The service role is an IAM role that Amazon EMR assumes to provision resources and perform service-level actions with other AWS services.

- Choose an existing service role — Select a default service role or a custom role with IAM
- Create a service role — Let Amazon EMR create a new service role so that you

Summary Info

Uniform instance groups

Primary (r8g.xlarge), Core (r8g.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration

Core size: 2 instances

Networking - *required*

VPC

vpc-0786984ef...

Subnet

subnet-0372e2...

Primary node security group

sg-025ad1917c...

Core node security group

us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster

aws | Search [Alt+S] | United States (Ohio) | RAustin2000 (40

Amazon EMR > EMR on EC2: Clusters > Create cluster

Amazon EC2 key pair for SSH to the cluster Info

aws_cloud | Browse | Create key pair

Identity and Access Management (IAM) roles - *required* Info

Choose or create a service role and instance profile for the EC2 instances in your cluster.

Amazon EMR service role Info

The service role is an IAM role that Amazon EMR assumes to provision resources and perform service-level actions with other AWS services.

- Choose an existing service role — Select a default service role or a custom role with IAM policies attached so that your cluster can interact with other AWS services.
- Create a service role — Let Amazon EMR create a new service role so that you can grant and restrict access to resources in other AWS services.

Service role

EMR_DefaultRole

EC2 instance profile for Amazon EMR

Summary Info

Uniform instance groups

Primary (r8g.xlarge), Core (r8g.xlarge)

Cluster scaling and provisioning - *required*

Provisioning configuration

Core size: 2 instances

Networking - *required*

VPC

vpc-0786984ef...

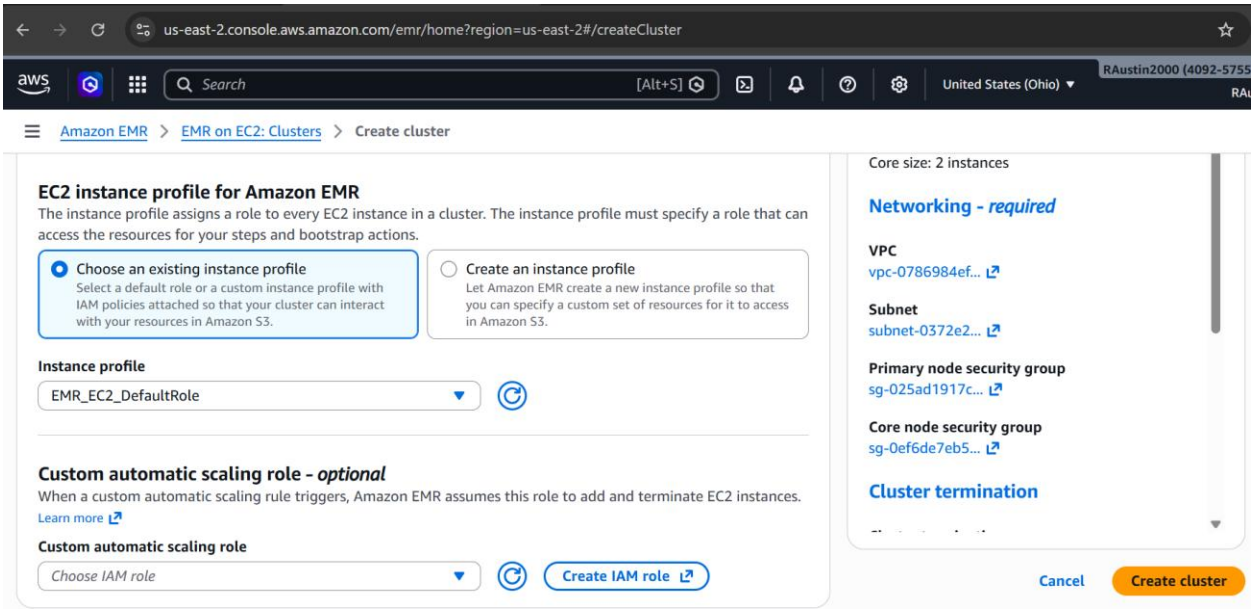
Subnet

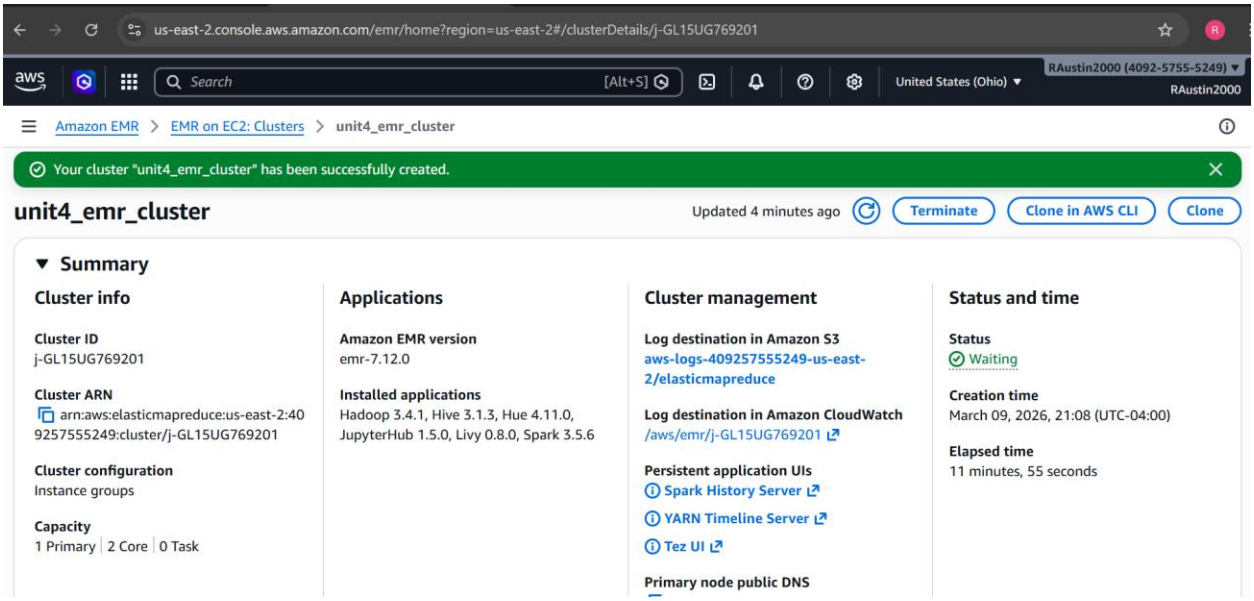
subnet-0372e2...

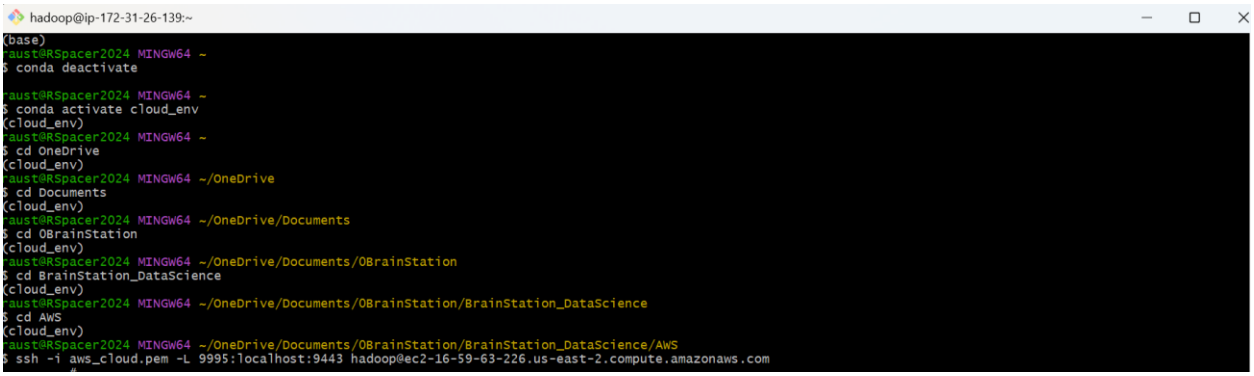
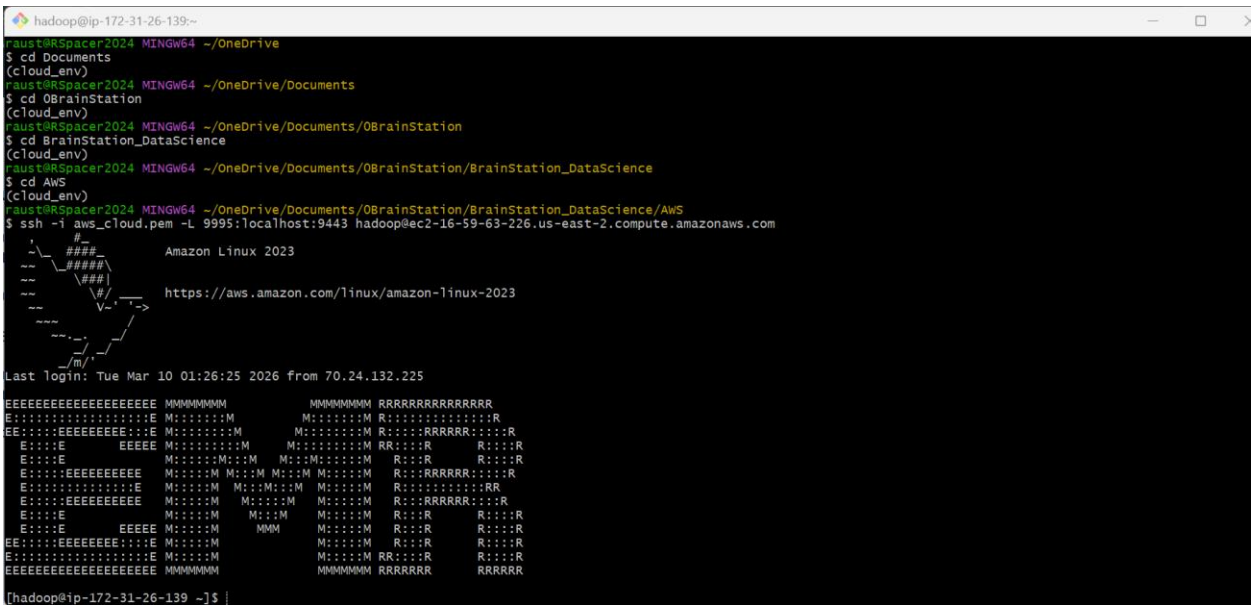
Primary node security group

sg-025ad1917c...

Core node security group


us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/createCluster
Search
[Alt+S]
United States (Ohio)
RAustin2000 (4092-5755
Amazon EMR > EMR on EC2: Clusters > Create cluster
EC2 instance profile for Amazon EMR
The instance profile assigns a role to every EC2 instance in a cluster. The instance profile must specify a role that can access the resources for your steps and bootstrap actions.
Choose an existing instance profile
Select a default role or a custom instance profile with IAM policies attached so that your cluster can interact with your resources in Amazon S3.
Create an instance profile
Let Amazon EMR create a new instance profile so that you can specify a custom set of resources for it to access in Amazon S3.
Instance profile
EMR_EC2_DefaultRole
Custom automatic scaling role - optional
When a custom automatic scaling rule triggers, Amazon EMR assumes this role to add and terminate EC2 instances.
Learn more
Custom automatic scaling role
Choose IAM role
Create IAM role
Core size: 2 instances
Networking - required
VPC
vpc-0786984ef...
Subnet
subnet-0372e2...
Primary node security group
sg-025ad1917c...
Core node security group
sg-0ef6de7eb5...
Cluster termination
Cancel
Create cluster


us-east-2.console.aws.amazon.com/emr/home?region=us-east-2#/clusterDetails/j-GL15UG769201
Search
[Alt+S]
United States (Ohio)
RAustin2000 (4092-5755-5249)
RAustin2000
Amazon EMR > EMR on EC2: Clusters > unit4_emr_cluster
Your cluster "unit4_emr_cluster" has been successfully created.
unit4_emr_cluster
Updated 4 minutes ago
Terminate
Clone in AWS CLI
Clone
Summary
Cluster info
Cluster ID
j-GL15UG769201
Cluster ARN
arn:aws:elasticmapreduce:us-east-2:409257555249:cluster/j-GL15UG769201
Cluster configuration
Instance groups
Capacity
1 Primary | 2 Core | 0 Task
Applications
Amazon EMR version
emr-7.12.0
Installed applications
Hadoop 3.4.1, Hive 3.1.3, Hue 4.11.0, JupyterHub 1.5.0, Livy 0.8.0, Spark 3.5.6
Cluster management
Log destination in Amazon S3
aws-logs-409257555249-us-east-2/elasticmapreduce
Log destination in Amazon CloudWatch
/aws/emr/j-GL15UG769201
Persistent application UIs
Spark History Server
YARN Timeline Server
Tez UI
Primary node public DNS
Status and time
Status
Waiting
Creation time
March 09, 2026, 21:08 (UTC-04:00)
Elapsed time
11 minutes, 55 seconds

APPENDIX II

Screenshots for connecting to the head node of the cluster using SSH.